Reflexion: Attempt Scaling and Diminishing Returns, by Hand

Why each retry costs the same but buys less —
the geometric decay of marginal quality, counted by hand.

What this document does. It takes the **Reflexion** loop from Module 2.10 — *generate* → *critique* → *revise*, repeat until good enough or out of budget — and prices the one decision every Reflexion agent must make: *how many attempts?* We fix a single per-attempt success probability p, derive the chance of succeeding within K attempts as $1-(1-p)^K$ *by hand*, and show the marginal gain decaying geometrically while the cost per attempt stays flat. Every number below is reproduced by the Python program in Appendix A. The module's headline guidance — "attempt 1→2 helps a lot, 2→3 less, **cap at 3**" — falls straight out of the arithmetic; we derive exactly where the cap comes from.

Where this sits. This is **Topic 2 of Module 2.10** (Planning Patterns) — the quantitative spine under the Reflexion "cost ★★□□" rating and the diminishing-returns callout. Its companions: Module 2.10 Topic 1 (ReAct vs Tree-of-Thoughts — spending calls on *search* instead of *retries*) and Module 0.0 Topic 1 (the cost of the ReAct loop — the linear baseline).

Concepts covered, in order: an attempt as 2–3 calls · linear cost $K\,c_{\text{attempt}}$ · success within K as $1-(1-p)^K$ (derived from the complement) · marginal gain $p(1-p)^{K-1}$ (geometric decay) · expected attempts $1/p$ · the quality-floor stop rule · why Reflexion needs a clear success signal.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: one task, retried until it passes

A Reflexion agent does its first attempt, has a critic score it, and — if the score is below the bar — revises and tries again. The module's own loop is exactly this:

```
def route(state): # from the module's reflexion.py
    latest = state["critiques"][-1]
    if latest.score >= 0.85: return "__end__" # quality bar -> ship
    if state["attempt"] >= 3: return "__end__" # budget cap -> stop
    return "draft" # revise: another attempt
```

Each **attempt** is a small bundle of LLM calls: a *generate* (or revise), a *critique*, and — counting the revise as its own call — roughly 2–3 calls. We will use 3. The cost of running K attempts is therefore dead simple: linear.

$$\text{Cost}(K) \;=\; K \cdot c_{\text{attempt}}. \tag{1}$$

The quantities. Round, hand-followable numbers; the mechanics are identical at any p.

Quantity	Symbol	Value here
Per-attempt success probability	p	0.40
Calls per attempt (gen + critique + revise)	—	3
Tokens read per call (flat)	T_{in}	2,000
Tokens written per call	T_{out}	300
Input price	p_{in}	\$3 / 1M tokens
Output price	p_{out}	\$15 / 1M tokens
Quality floor (stop threshold)	τ	0.15

Notation. "Success" means the attempt clears the critic's bar. We model attempts as *independent* with the same probability p — an idealisation we will be honest about at the end. P_K is the probability of having succeeded *by* attempt K.

Intuition

Reflexion is rolling a weighted die until it lands on "good enough." Every roll costs the same handful of LLM calls. But each roll only matters if *all the earlier rolls missed* — and the chance of that shrinks fast. So you pay a flat price per roll while the value of the roll decays geometrically. That gap between flat cost and decaying value is the entire story of diminishing returns.

2 Step 1 — success within K attempts

We want P_K, the probability that at least one of K attempts succeeds. Attacking "at least one" directly means summing many cases; the clean move is the **complement**. The only way to *fail overall* is for *every* attempt to fail. A single attempt fails with probability $(1-p)$, and the attempts are independent, so K failures in a row has probability $(1-p)^K$. Therefore

$$\boxed{P_K \;=\; 1-(1-p)^K}. \tag{2}$$

With $p = 0.4$, so $(1-p) = 0.6$:

K	$(1-p)^K = 0.6^K$	$P_K = 1 - 0.6^K$	**cumulative**
1	0.600	0.400	40.0%
2	0.360	0.640	64.0%
3	0.216	0.784	78.4%
4	0.130	0.870	87.0%
5	0.078	0.922	92.2%

Sample check, $K = 3$: $0.6^3 = 0.216$, so $P_3 = 1 - 0.216 = 0.784$. ✓ Three attempts at a 40% success rate clear the task 78% of the time — a big lift over one attempt's 40%. But look at the *steps between rows*: +0.240, then +0.144, then +0.086. The gains are shrinking, even though each row cost exactly one more attempt.

3 Step 2 — the marginal gain decays geometrically

That shrinking is not an accident; it has a closed form. The gain from the K-th attempt is the jump $P_K - P_{K-1}$:

$$\begin{aligned} P_K - P_{K-1} &= \big[1-(1-p)^K\big] - \big[1-(1-p)^{K-1}\big] \\ &= (1-p)^{K-1} - (1-p)^K \\ &= (1-p)^{K-1}\big[\,1-(1-p)\,\big] = \boxed{\,p\,(1-p)^{K-1}\,}. \end{aligned} \tag{3}$$

This is exactly the chance that attempts $1 \dots K-1$ all failed *and* attempt K succeeds — a clean probabilistic reading. It is a **geometric sequence** with ratio $(1-p) = 0.6$: each attempt's marginal gain is 60% of the previous one's.

Attempt K	**marginal gain** $p(1-p)^{K-1}$	**vs. previous**
1	0.400	—
2	0.240	×0.6
3	0.144	×0.6
4	0.086	×0.6
5	0.052	×0.6

Mini-example — this operation in isolation

The decay is just repeated multiplication by 0.6. Start at $p = 0.40$. Attempt 2 adds $0.40 \times 0.6 = 0.24$; attempt 3 adds $0.24 \times 0.6 = 0.144$; attempt 4 adds $0.144 \times 0.6 = 0.0864$. ✓ Each retry hands you 60% of what the last one did. By attempt 4 you are buying less than a 9-point improvement for the same three LLM calls that attempt 1 spent buying 40 points.

Intuition

The expected number of attempts to your *first* success is $1/p$. With $p = 0.4$ that is 2.5 attempts. So "cap at 3" is not arbitrary: it sits just past the average solve point, catching the typical task while refusing to bankroll the unlucky tail. The geometric tail 0.6^K never reaches zero — there is always *some* task that ten retries would have fixed — but each extra attempt past 3 rescues a vanishing slice for full price.

4 Step 3 — flat cost meets decaying gain

Now put cost beside value on one line. Each call is the flat $c_{\text{call}} = 2000 \cdot 3{\times}10^{-6} + 300 \cdot 15{\times}10^{-6} = \$0.0105$, and an attempt is 3 calls, so $c_{\text{attempt}} = 3 \times \$0.0105 = \$0.0315$.

K	cum. cost $K\,c_{\text{attempt}}$	P_K	marginal gain	gain per \$
1	\$0.0315	0.400	0.400	12.70
2	\$0.0630	0.640	0.240	7.62
3	\$0.0945	0.784	0.144	4.57
4	\$0.1260	0.870	0.086	2.74
5	\$0.1575	0.922	0.052	1.65

The cost column is a straight line (Equation (1)); the marginal-gain column is the geometric decay (Equation (3)). "Gain per \$" — the efficiency of the marginal attempt — falls by the same factor 0.6 each row: $12.70 \to 7.62 \to 4.57$. Sample check, $K = 3$: cost $= 3 \times \$0.0315 = \$0.0945$, gain per \$ $= 0.144/0.0315 = 4.57$. ✓ You are paying the same \$0.0315 each time to buy steadily cheaper improvement.

5 Step 4 — the stop rule, derived

When should the loop quit? A flat cost against a decaying gain has a crossing point. The clean formulation is a **quality floor** τ: keep retrying only while the next attempt's marginal gain clears τ. Stop at the first K with

$$p\,(1-p)^{K-1} \;<\; \tau.$$

Solve for K. Divide by p, take logs (and remember $\ln(1-p) < 0$ flips the inequality):

$$\begin{aligned} (1-p)^{K-1} &< \tau/p \\ (K-1)\ln(1-p) &< \ln(\tau/p) \\ K-1 &> \frac{\ln(\tau/p)}{\ln(1-p)} \quad\Longrightarrow\quad \boxed{K^\star = 1 + \frac{\ln(\tau/p)}{\ln(1-p)}}. \end{aligned} \tag{4}$$

Plug in $p = 0.4$, $\tau = 0.15$: $\tau/p = 0.375$, $\ln(0.375) = -0.981$, $\ln(0.6) = -0.511$, so $K^\star = 1 + (-0.981)/(-0.511) = 1 + 1.92 = 2.92$. Round up: stop at $\mathbf{K = 3}$. ✓ The marginal gains confirm it directly — attempt 3 adds 0.144 (above the 0.15 floor only just; attempt 4 adds 0.086, below it). **The module's "cap at 3" is this inequality.** Set a laxer floor $\tau = 0.10$ and $K^\star$ moves to 4; a stricter $\tau = 0.20$ pulls it to 3 as well.

> **Watch out**
>
> The dollar value of solving a real task usually dwarfs \$0.03 of compute, so a *pure* cost stop rule would say "retry forever." That is the wrong lesson. The true ceiling on Reflexion is not the cost — it is that the gains plateau *and then regress*: the module's *over-reflection* pitfall, where attempt 5 is worse than attempt 2 because a same-model critic starts rubber-stamping. The quality floor τ stops you at the plateau, before the regression. Cap at 3, and keep *best-so-far*, not the last attempt.

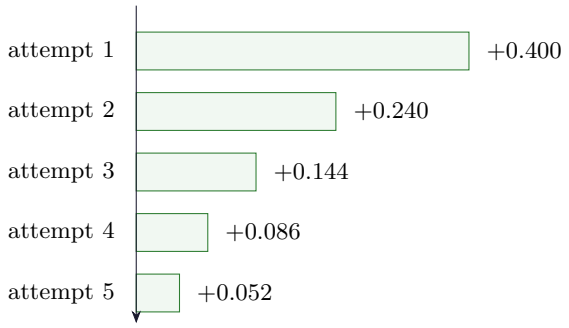
6 Step 5 — when Reflexion works at all

The whole derivation assumed one thing: that the critic can actually tell success from failure. If the score is reliable, retries climb the $1-(1-p)^K$ curve. If it is not, you are looping on noise — spending $K\,c_{\text{attempt}}$ to walk in a circle.

- **Works:** code (run the tests — pass/fail is ground truth), structured output (schema validates or it does not), anything with a deterministic check. Here the success signal *is* the environment, and p is real.
- **Breaks:** open-ended writing or strategy, where "good enough" is the same model's opinion. The critic and the author share blind spots, so p is not what you think and the 0.6-decay assumption quietly fails.

> **Intuition**
>
> Reflexion converts a *verifier* into a *generator*: if you can cheaply check an answer, you can keep trying until one checks out, and $1-(1-p)^K$ tells you how many tries that takes. The pattern is exactly as strong as your check. No check, no Reflexion — just an expensive way to rephrase the same first draft.

7 The decay, drawn



Each bar is the marginal quality bought by that attempt — $p(1-p)^{K-1}$. Every bar is 0.6 as long as the one above it: the geometric decay of Equation (3), made visible. The cost of each attempt, by contrast, would be five *identical* bars. Flat cost, shrinking value — that is the picture of diminishing returns, and the reason the loop has a cap.

8 Reflexion cheat-sheet

Cost of K attempts	$\mathrm{Cost}(K) = K\, c_{\text{attempt}}$ (linear)
Success within K	$P_K = 1 - (1-p)^K$
Marginal gain of attempt K	$P_K - P_{K-1} = p\,(1-p)^{K-1}$
Decay ratio between attempts	$(1-p)$ (here 0.6)
Expected attempts to first success	$1/p$ (here 2.5)
Stop rule (quality floor τ)	$K^\star = 1 + \dfrac{\ln(\tau/p)}{\ln(1-p)}$
Hard precondition	a *reliable* success signal (tests / schema)

9 An honest word about these numbers

The success rate $p = 0.4$, the 3 calls per attempt, and the flat token sizes are illustrative round numbers, chosen so the table is followable by hand. Two modelling assumptions are stronger than they look. First, **independence**: real revisions are *not* independent draws — a good critique makes attempt $K{+}1$ better than a fresh roll, which helps; a yes-man critic makes it worse, which hurts. So the true curve can beat or fall below $1-(1-p)^K$. Second, a **constant** p: in practice p drifts attempt-to-attempt. What is *not* illustrative is the structure: when retries are even roughly independent, the success curve is $1-(1-p)^K$, its marginal gains decay geometrically by the factor $(1-p)$, and the cost stays stubbornly linear. That mismatch — flat cost, geometric value — is why every Reflexion loop ships with an attempt cap, and why the cap lands around 3 for any sensible p and floor.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the success curve, the geometric marginal gains, the cost comparison, and the stop-rule $K^\star$; change `p`, `tau`, or the prices to explore your own loop.

```python
# reflexion_scaling.py -- reproduces every number in "Reflexion: Attempt Scaling".
import math
P_IN, P_OUT = 3e-6, 15e-6
T_IN, T_OUT = 2000, 300
c_call = T_IN * P_IN + T_OUT * P_OUT # = $0.0105
c_attempt = 3 * c_call # gen + critique + revise = $0.0315

p = 0.4 # per-attempt success probability
tau = 0.15 # quality floor: stop when next attempt's gain < tau

def P(K): # success within K attempts = 1 - (1-p)^K
    return 1 - (1 - p) ** K

def marginal(K): # gain of the K-th attempt = p*(1-p)^(K-1)
    return p * (1 - p) ** (K - 1)

# --- success curve + marginal gain ---
print(" K (1-p)^K P_K marginal cost gain/$")
for K in range(1, 6):
    print(f" {K} {(1-p)**K:6.3f} {P(K):5.3f} {marginal(K):6.3f} "
          f"${K*c_attempt:6.4f} {marginal(K)/c_attempt:6.2f}")
```

```python

print(f"\nexpected attempts to first success = 1/p = {1/p}")
print(f"c_call = ${c_call:.4f} c_attempt = ${c_attempt:.4f}")

# --- stop rule: smallest K with marginal gain < tau ---
Kstar = 1 + math.log(tau / p) / math.log(1 - p)
print(f"K* (continuous) = {Kstar:.2f} -> cap at K = {math.ceil(Kstar)}")

# Expected output:
# K (1-p)^K P_K marginal cost gain/$
# 1 0.600 0.400 0.400 $0.0315 12.70
# 2 0.360 0.640 0.240 $0.0630 7.62
# 3 0.216 0.784 0.144 $0.0945 4.57
# 4 0.130 0.870 0.086 $0.1260 2.74
# 5 0.078 0.922 0.052 $0.1575 1.65
#
# expected attempts to first success = 1/p = 2.5
# c_call = $0.0105 c_attempt = $0.0315
# K* (continuous) = 2.92 -> cap at K = 3
```

— end of document —